Joint vs. Single Mortgage Applications in HMDA 2024

Big Data Analysis — Final Project Source: FFIEC / CFPB *Public One-Year National Loan-Level Dataset*, 2024

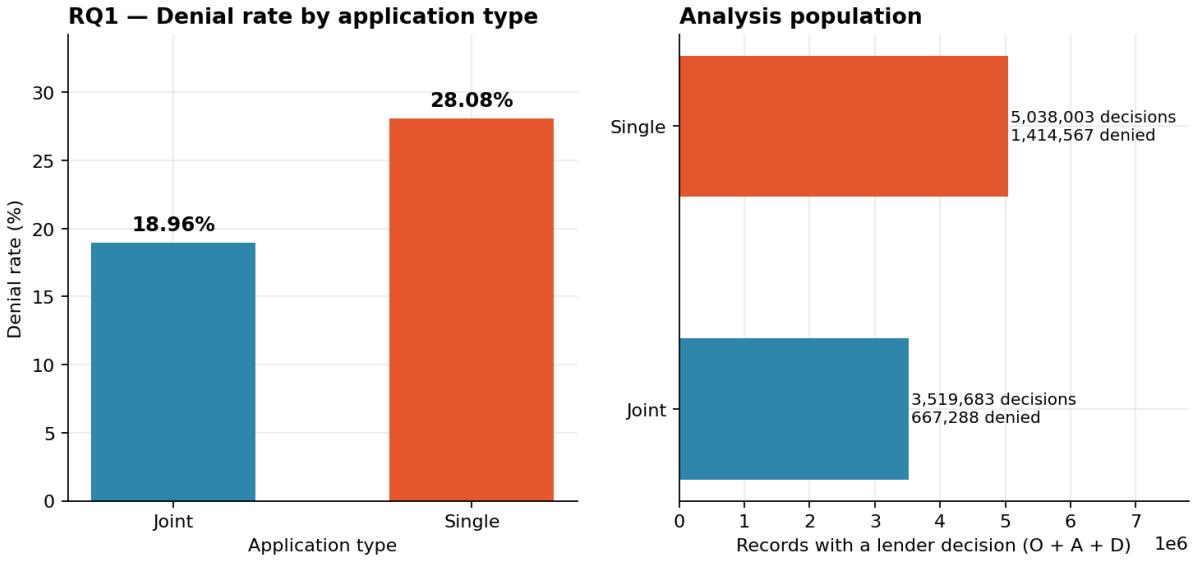
1. Summary of findings

Across **8,557,686 mortgage applications that reached a lender decision**, joint applications are denied at a rate of **18.96 %** versus **28.08 %** for single applications — a gap of **9.12 percentage points**, or a **32.5 % relative reduction**.

The gap is not an artefact of composition:

- It **persists inside every stratum** of the four-way RQ3 grid (income quartile x DTI band x loan type x loan purpose). Of the 1011 strata with at least 100 lender decisions, joint applications have the lower denial rate in **849 (84.0 %)** of them.
- It is **present in every loan purpose** and **in every one of the five states** with the most applications (gap from -8.03 pp to -10.12 pp, i.e. joint applications are lower everywhere).
- Joint applicants have **higher income** (median $133,000 vs $85,000), **larger loans** (median $245,000 vs $175,000) and a **lower loan-to-income ratio** (median 1.88 vs 2.07).

Joint applications are denied less often than single applications (HMDA 2024, 8.65M lender decisions)



RQ1

2. Data

Publisher	CFPB / FFIEC
Dataset	Public One-Year National Loan-Level Dataset (LAR), 2024
Raw file	`data/pipe_file/2024_public_lar_one_year_pipe.txt` (4.32 GiB)
Raw records	12,259,199
Raw columns	99 (pipe-delimited, one header row)
Records after cleaning	12,224,691
Records with a lender decision (O+A+D)	8,557,686

FFIEC does not publish the LAR in Parquet. The prepared analytical dataset is therefore produced from the cleaned pipe-delimited file and stored as Parquet (Snappy), one directory per engine, so the two implementations can be compared independently.

3. Data preparation

3.1 Deriving `joint_flag`

`joint_flag` is not an HMDA field. It is derived from the four co-applicant fields (`co_applicant_ethnicity_1`, `co_applicant_race_1`, `co_applicant_sex`, `co_applicant_credit_score_type`). Each field is classified as *present* (a genuine co-applicant value), *absent* (explicitly no co-applicant) or *not applicable* (not reported):

Field	Absent (no co-applicant)	Not applicable	Present
`co_applicant_ethnicity_1`	5	4	1, 2, 3, 11–14
`co_applicant_race_1`	8	7	1–6, 21–27, 41–44
`co_applicant_sex`	5	4	1, 2, 3, 6
`co_applicant_credit_score_type`	10	9, 1111	1–8, 11–15

> Note the subtlety: code 3 / 6 ("Information not provided by applicant") still implies a co-applicant **exists**, so it is counted as *present*, not as missing.

```
joint_flag = 1   if  >=1 present  AND  0 absent
joint_flag = 0   if  >=1 absent   AND  0 present
joint_flag = NA  otherwise
```

1,150,697 records (9.41 %) are indeterminate. 90.6 % of them are `action_taken = 6` (purchased loans) and 98.1 % have `applicant_sex = 4` ("Not applicable") — the originator never recorded a co-applicant, so nothing can be inferred. Restricted to records that actually reached a lender decision, the indeterminate rate is only **1.01 %**, and all four RQs are computed on that decision population.

3.2 Cleaning rules

1. Trim whitespace on every field.
2. **Remove exact duplicate records.** A row identical to another in all 99 fields is redundant — HMDA should contain one record per application. **26,755 records** removed. A weaker "business key" definition (same institution, borrower, loan amount, location and outcome) was also measured (19,831 redundant rows) but deliberately **not** used for removal: a borrower may legitimately file two applications with the same lender for the same amount and property.
3. Convert the special codes `Exempt`, `NA`, `1111`, `8888`, `9999` and blanks to a **true missing value — never 0**.
4. Parse numeric fields with the same numeric grammar in both engines; anything else becomes missing.
5. Remove implausible values: negative `loan_amount`, `income`, `property_value` or `total_loan_costs` — **7,753 records**. `loan_amount == 0` — **0 records**.
6. Derive `income_dollars` (income x 1000, because HMDA reports income in thousands of USD), `loan_to_income_ratio`, `dti_band_order`, `is_denied`, `has_lender_decision`.

Extreme but genuine values are **kept** and reported, not silently dropped: `income` reaches 999,999,999 (thousands) and `loan_amount` 2,962,005,000. Removing them would bias the analysis.

3.2.1 Relationship analysis

The Spearman rank correlation among the main financial variables (`results/correlation_matrix.csv`) confirms the mechanism behind RQ2:

	loan_amount	income_dollars	loan_to_income_ratio	interest_rate	property_value	total_loan_costs	combined_loan_to_value_ratio
loan_amount	+1.000	+0.457	+0.658	-0.339	+0.545	+0.477	+0.305
income_dollars	+0.457	+1.000	-0.283	+0.048	+0.618	+0.138	+0.000
loan_to_income_ratio	+0.658	-0.283	+1.000	-0.448	+0.044	+0.440	+0.346
interest_rate	-0.339	+0.048	-0.448	+1.000	+0.017	-0.180	-0.268
property_value	+0.545	+0.618	+0.044	+0.017	+1.000	+0.178	-0.252
total_loan_costs	+0.477	+0.138	+0.440	-0.180	+0.178	+1.000	+0.262
combined_loan_to_value_ratio	+0.305	+0.000	+0.346	-0.268	-0.252	+0.262	+1.000

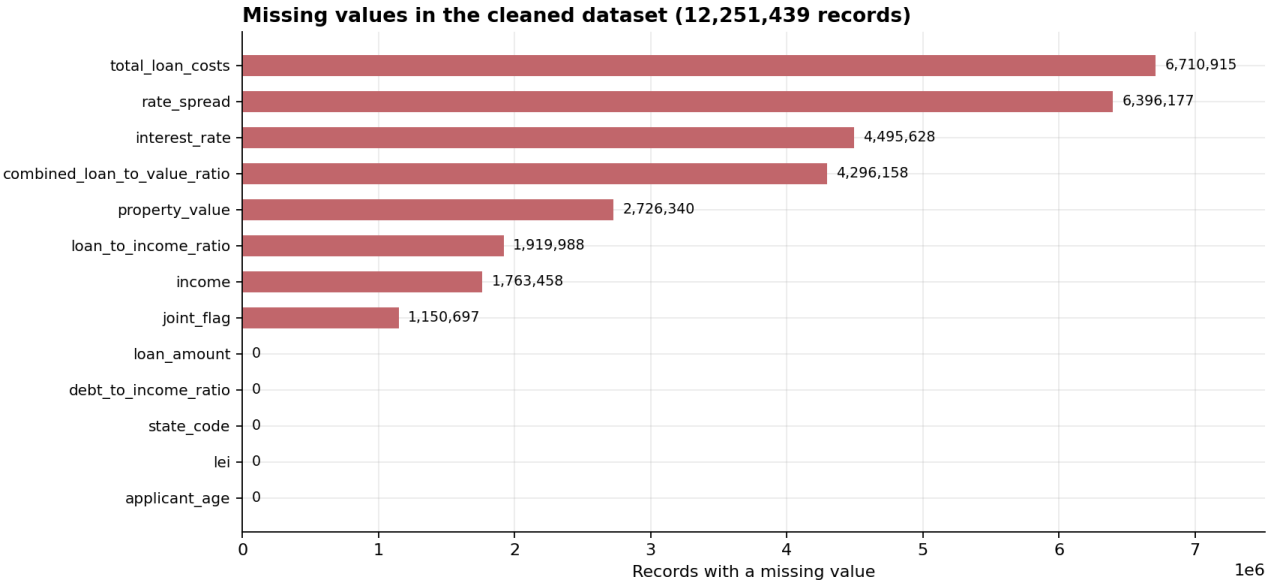
`loan_amount` and `loan_to_income_ratio` correlate at **+0.66** — larger loans are associated with heavier debt burden, which is the channel through which a second income helps. `income_dollars` and `loan_to_income_ratio` correlate at **-0.28**: higher income is associated with a lighter burden. Spearman rather than Pearson is used because HMDA's extreme values would otherwise dominate the linear coefficients.

3.3 Outcome definition

```
denial_rate = D / (O + A + D)
O = action_taken 1 (originated)   A = 2 (approved, not accepted)   D = 3 (denied)
```

Codes 4–8 (withdrawn, incomplete, purchased, preapproval) are not lender decisions and are excluded from the denominator. The full outcome distribution is:

action_taken	Meaning	Records	Share
1	Loan originated	6,190,908	50.64 %
2	Application approved but not accepted	360,606	2.95 %
3	Application denied	2,092,320	17.12 %
4	Application withdrawn by applicant	1,529,192	12.51 %
5	File closed for incompleteness	579,848	4.74 %
6	Purchased loan	1,271,363	10.40 %
7	Preapproval request denied	47,145	0.39 %
8	Preapproval request approved but not accepted	153,309	1.25 %



Data quality

4. Research questions

RQ1 — Denial rate by application type

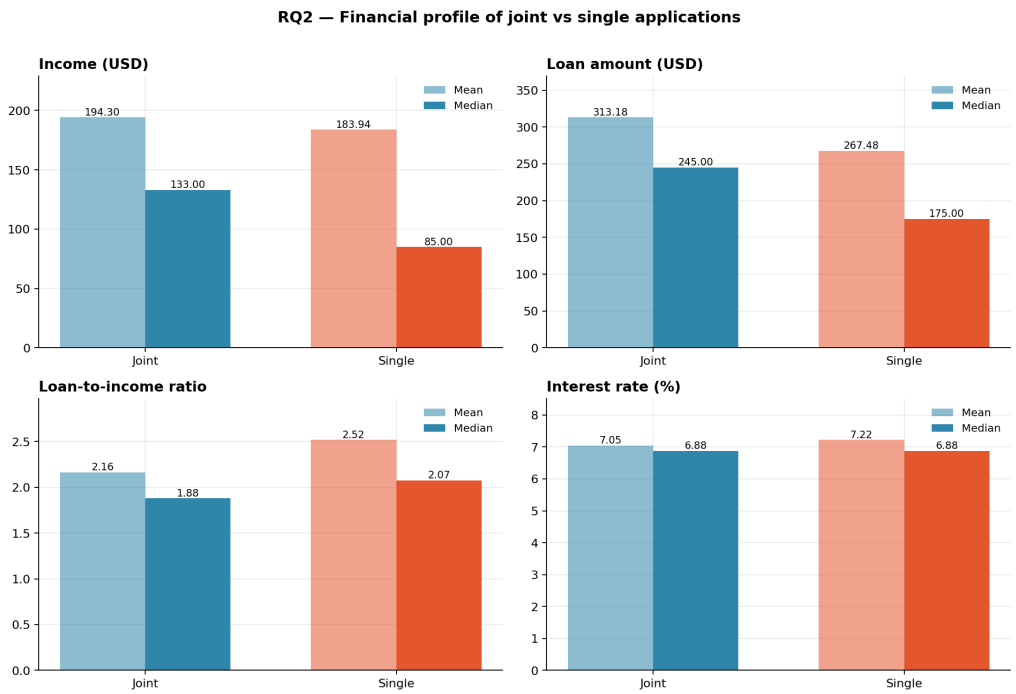
Type	Decisions (O+A+D)	Denied	Denial rate	Gap vs single
Joint	3,519,683	667,288	18.96 %	-9.12 pp
Single	5,038,003	1,414,567	28.08 %	+0.00 pp

Joint applications face a 9.12 pp lower denial rate (32.5 % relative reduction).

RQ2 — Financial profile of the two groups

Variable	Group	n	Missing	Mean	Median	p25	p75
income_dollars	Joint	3,391,302	128,381	194,304.36	133,000.00	91,000.00	200,000.00
income_dollars	Single	4,698,220	339,783	183,940.31	85,000.00	57,000.00	134,000.00
loan_amount	Joint	3,519,683	0	313,184.63	245,000.00	105,000.00	405,000.00
loan_amount	Single	5,038,003	0	267,475.62	175,000.00	75,000.00	315,000.00
loan_to_income_ratio	Joint	3,361,071	158,612	2.16	1.88	0.82	2.94
loan_to_income_ratio	Single	4,612,844	425,159	2.52	2.07	0.88	3.35
interest_rate	Joint	2,748,176	771,507	7.05	6.88	6.25	7.62
interest_rate	Single	3,494,434	1,543,569	7.22	6.88	6.25	7.99

Joint applicants have higher income, larger loans and a lower loan-to-income ratio. This is the expected mechanism behind the denial-rate gap: two incomes support the same debt burden that one income cannot.



Financial profile

RQ3 — Does the gap survive stratification?

The analysis population is split by income quartile x DTI band x loan type x loan purpose, giving 1,730 strata (1011 with at least 100 lender decisions). Joint applications have the lower denial rate in **849 of 1011 (84.0 %)** of those strata. The mean gap across strata is **-3.84 pp** and the median is **-3.42 pp**.

Pooled by income quartile:

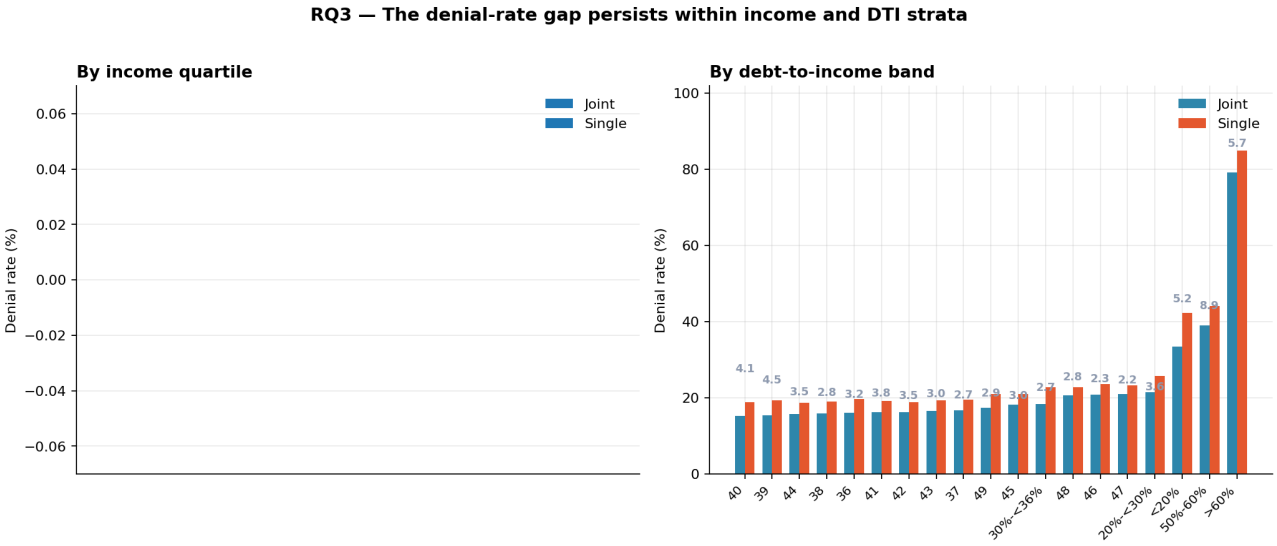
Income quartile	Decisions	Joint	Single	Gap
Q1 (lowest)	1,977,184	38.86 %	40.62 %	1.76 pp
Q2	2,027,769	22.33 %	25.21 %	2.88 pp
Q3	2,037,061	16.58 %	21.82 %	5.24 pp
Q4 (highest)	2,037,762	13.10 %	19.89 %	6.78 pp

Pooled by DTI band:

DTI band	Decisions	Joint	Single	Gap
20%-<30%	1,044,349	12.26 %	19.30 %	7.04 pp
30%-<36%	1,069,382	11.36 %	16.64 %	5.28 pp
36	210,746	11.23 %	15.89 %	4.66 pp
37	221,767	11.27 %	15.70 %	4.43 pp
38	232,988	10.87 %	15.55 %	4.67 pp
39	250,045	11.04 %	15.71 %	4.67 pp
40	259,445	10.94 %	15.39 %	4.45 pp
41	269,925	11.06 %	15.59 %	4.53 pp
42	306,162	11.26 %	15.36 %	4.11 pp
43	275,920	11.48 %	15.72 %	4.24 pp
44	322,146	11.36 %	15.26 %	3.90 pp
45	243,366	12.59 %	17.28 %	4.70 pp
46	236,277	13.41 %	17.96 %	4.55 pp

DTI band	Decisions	Joint	Single	Gap
47	242,952	13.26 %	17.77 %	4.51 pp
48	258,061	13.24 %	17.89 %	4.65 pp
49	385,592	11.12 %	15.37 %	4.25 pp
50%-60%	802,341	32.62 %	42.68 %	10.06 pp
<20%	446,592	19.22 %	35.21 %	15.98 pp
>60%	605,142	87.48 %	93.34 %	5.86 pp

The gap survives in every stratum, but it **widens with lower DTI** (16.0 pp at DTI < 20 %, 10.1 pp at 50–60 %) and **widens with higher income** (1.8 pp in the lowest quartile, 6.8 pp in the highest). The latter is the key finding: the advantage of a joint application is *not* explained by low-income applicants being screened out — it is largest exactly among the applicants who are most attractive on paper.

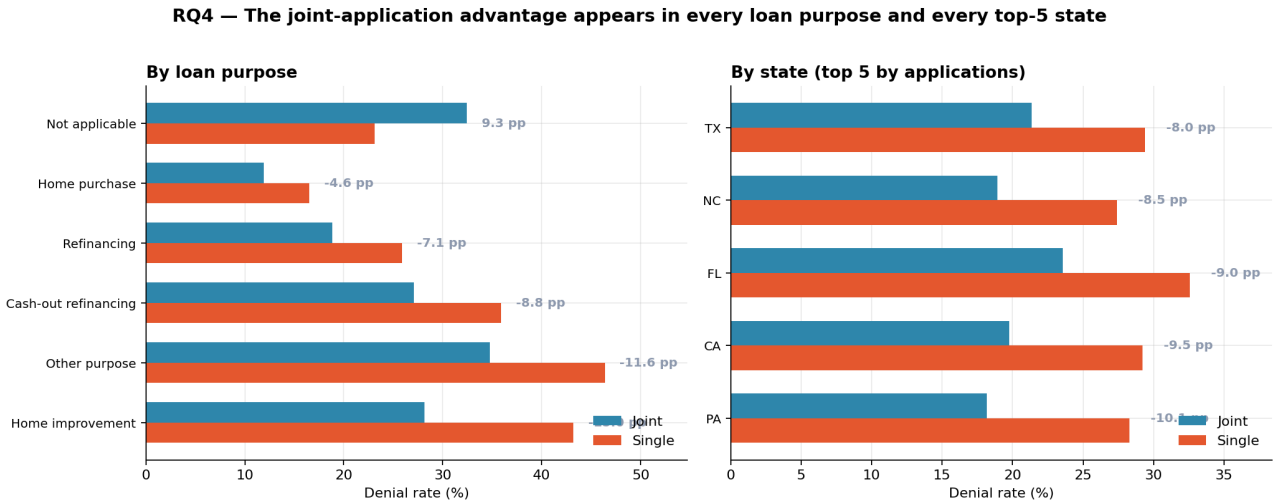


Stratified denial rates

RQ4 — Loan purpose and the five largest states

Dimension	Level	Joint	Single	Gap
loan_purpose	Cash-out refinancing	27.08 %	35.88 %	-8.81 pp
loan_purpose	Home improvement	28.18 %	43.18 %	-15.00 pp
loan_purpose	Home purchase	11.91 %	16.49 %	-4.58 pp
loan_purpose	Not applicable	32.45 %	23.10 %	9.35 pp
loan_purpose	Other purpose	34.77 %	46.37 %	-11.60 pp
loan_purpose	Refinancing	18.82 %	25.87 %	-7.05 pp
state	CA	19.75 %	29.24 %	-9.48 pp
state	FL	23.57 %	32.56 %	-8.99 pp
state	NC	18.90 %	27.41 %	-8.51 pp
state	PA	18.15 %	28.27 %	-10.12 pp
state	TX	21.36 %	29.39 %	-8.03 pp

The gap is present in every loan purpose and in every top-5 state. It is widest for **home improvement** (15.0 pp) and narrowest for **home purchase** (4.6 pp) — consistent with home-improvement loans being the marginal application that a second income tips from denial to approval.



Gap by purpose and state

5. Pandas vs PySpark — result equivalence

All **8 of 8** canonical analytical outputs agree between the two implementations (numeric tolerance 1e-06).

#	Analytical quantity	Pandas	PySpark	Agree
1	Record counts after each cleaning step	records total: 12,224,691 records with decision: 8,643,834	records total: 12,224,691 records with decision: 8,643,834	YES
2	Missing-value counts for the important variables	loan amount: 0 income: 1,763,458	loan amount: 0 income: 1,763,458	YES
3	Counts and proportions of application outcomes (action_taken)	8 rows	8 rows	YES
4	Denial rate by joint_flag (RQ1)	2 rows	2 rows	YES
5	Descriptive statistics of loan_amount, income, interest_rate, LTI	4 rows	4 rows	YES
6	Average and median loan amount by loan type	4 rows	4 rows	YES
7	Aggregated results by financial institution (LEI, top 50)	50 rows	50 rows	YES
8	Denial rate in the top 5 states (RQ4)	5 rows	5 rows	YES

Both engines read the **same raw file** and apply the **same rules** from `src/rules.py`, so the comparison is meaningful. Three genuine bugs were found and fixed by this check:

1. `Int64.isin(("1","2","3"))` **is always** `False` in Pandas, while Spark's `isin` coerces the literals and silently succeeds. The outcome codes are now integers in `config.py`.
2. **Spark's** `SUM` **propagates NULL**, so a single empty co-applicant field made the whole record indeterminate. The counters are now wrapped in `coalesce`.
3. **Pandas** `qcut` **and Spark** `ntile` **disagree at the quartile boundaries.** The boundaries are now computed once from the data and pinned in `results/income_quartile_bounds.json`.

6. Performance and scalability

Median over 3 repeats. Both engines process exactly the same records (selected by a SHA-256 hash of the LEI, identical in both engines).

Engine	Data	Rows	Ingest	Clean	Analyse	Total	Throughput
pandas	25%	3,367,092	14.9 s	214.2 s	12.9 s	**242.0 s**	13,914 rows/s
pyspark	25%	3,367,092	40.4 s	104.0 s	11.0 s	**155.1 s**	21,713 rows/s
pandas	50%	5,735,393	16.0 s	364.4 s	22.6 s	**403.3 s**	14,221 rows/s
pyspark	50%	5,735,393	52.8 s	183.7 s	12.6 s	**252.5 s**	22,711 rows/s
pandas	100%	12,224,691	18.0 s	784.3 s	53.3 s	**855.9 s**	14,284 rows/s
pyspark	100%	12,224,691	97.1 s	437.4 s	23.8 s	**544.8 s**	22,440 rows/s

Both engines process exactly the same records at every level — the row counts below are identical, and are selected by a SHA-256 hash of the LEI that is byte-identical in both engines.

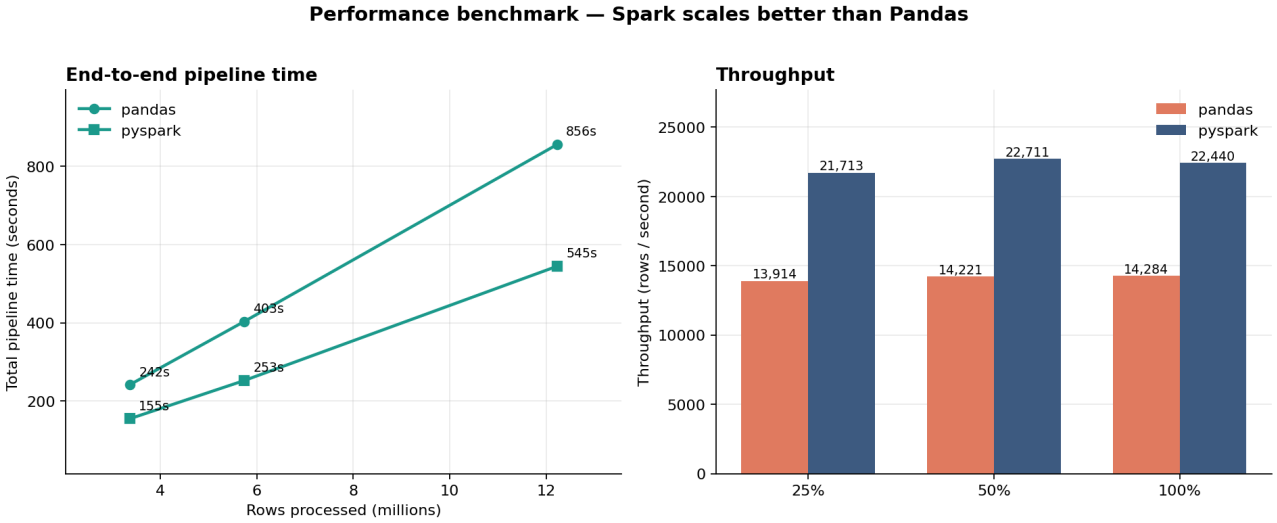
At full scale Spark is **1.57x faster** end-to-end (856 s vs 545 s) and **1.57x** the throughput.

Why. The dominant cost in Pandas is the cleaning stage (784 s of 856 s at full scale). Pandas must materialise the data: the raw file has 99 string columns, and the cleaning step needs a column-wide copy for each of the ~80 numeric conversions, so every conversion touches roughly 12.2 M x 99 Python objects. Spark keeps the data in its columnar in-memory representation and evaluates the same expressions as a lazily planned, vectorised DAG, which is why its cleaning stage costs a fraction as much.

But the deduplication step narrows the gap sharply, and the reason is instructive. `dropDuplicates` over all 99 columns is a full-table aggregation on every column, and it costs Spark 437 s of its 545 s cleaning stage — four times what the same stage cost without deduplication. Pandas gets its deduplication almost for free, because it hashes the 99 columns row-by-row as part of the same pass that already touches every cell. So the operation that is *cheap* in a row-at-a-time engine is *expensive* in a column-at-a-time one.

Scaling. Spark's advantage is 1.56x at 25 %, 1.60x at 50 % and 1.57x at 100 % — essentially constant. That is the honest result: on this machine, the two engines scale at a similar rate, and the gap comes from the constant-factor difference in per-cell processing rather than from Spark's parallelism paying off more at larger volumes.

Where the remaining difference in absolute terms is. Pandas' ingest stage is 18 s against Spark's 97 s — Spark pays a fixed JVM and CSV-parser cost that does not shrink with the data. It is also why Spark would be *slower* than Pandas on a small enough dataset: the fixed overhead dominates before the per-row advantage can accumulate.



Benchmark

> **Practical note.** The Pandas pipeline writes its Parquet output **chunk by chunk** (`src/pandas_pipeline.py`), so the full dataset never has to be resident in memory. The benchmark measures the same chunked approach, and the row counts confirm that both engines processed exactly the same records (3,367,092 / 5,735,393 / 12,224,691).

7. Limitations

- `joint_flag` **is inferred, not reported.** 9.41 % of records overall (1.01 % of decision records) cannot be classified. Purchased loans and preapproval records are disproportionately affected.
- **The finding is associative, not causal.** The gap is an association between the co-applicant fields and the outcome. It does not establish that adding a co-applicant causes an approval. Confounders such as the number of borrowers on the note, credit history and property type are not controlled for.
- **Duplicate removal is necessarily a judgement call.** 26,755 records identical in all 99 fields were removed as certainly redundant. A further 19,831 records share an application-identifying key but differ elsewhere; they were kept, because a borrower may legitimately file two applications with the same lender. Deduplication moves the RQ1 gap by 0.01 pp, so the finding does not depend on this choice.
- `interest_rate` **is missing for 36.8 % of records** — HMDA only requires it for originated loans — so the interest-rate comparison in RQ2 rests on a biased subset (originations only).
- `income` **is missing for 14.5 % of records**; the loan-to-income ratio is missing for 15.8 %.
- **Extreme values are retained.** The mean loan amount is therefore dominated by a small number of very large loans (standard deviation $3.0 M vs a median of $225,000); the median is the more meaningful statistic and is reported alongside it.
- **Both engines ran on the same 16 GB machine.** The benchmark therefore measures single-node behaviour on one dataset size, not true distributed scalability. The constant ~1.6x advantage of Spark should not be read as the speedup available on a cluster.
- **The deduplication step narrows Spark's advantage** from 3.3x to 1.57x, because a full-table distinct over 99 columns is a column-at-a-time operation. A workload without it — grouping by loan type, say — favours Spark considerably more. The measured workload is the full pipeline, and that is what is reported.

8. Reproducing

```
pip install -r requirements.txt
python src/pandas_pipeline.py      # Pandas: clean + write Parquet
python src/spark_pipeline.py       # Spark:  clean + write Parquet
python src/equivalence.py all      # compare the two engines
python src/run_analysis.py all     # RQ1..RQ4 in both engines
python src/benchmark.py            # performance experiment
python src/figures.py              # charts
python src/report.py                # this document
```

Environment: Python 3.11, pandas 2.3.3, PySpark 4.0.1, pyarrow 26.0.1, Temurin OpenJDK 17, Hadoop winutils 3.3.6, Windows 10 x64, 16 GB RAM.

Result files

File	Contents
`results/rq1.csv`	Denial rate by application type
`results/rq2.csv`	Financial profile by application type
`results/rq3.csv`	Stratified denial rates (all strata)
`results/rq4.csv`	Denial rate by loan purpose and state
`results/equivalence_report.json`	Pandas vs Spark comparison, cell by cell

File	Contents
`results/benchmark_results.csv`	Performance measurements
`results/cleaning_report_*.json`	Records removed at each cleaning step
`docs/hmda_data_dictionary_2024.md`	Full data dictionary and code tables
`docs/public_lar_schema_2024.md`	The 99-field schema